

Conectores y protocolos de acceso a bases de datos

2. Conectores y protocolos de acceso a bases de datos

El problema de partida: objetos contra tablas

¿Cómo habla tu programa Java, con sus clases y objetos, con una base de datos que solo entiende tablas y filas?

En Java

```
class Editorial {  
    String nombre;  
    List<Libro> libros;  
}
```

En la base de datos: dos tablas con clave foránea

```
CREATE TABLE editorial (id INT PRIMARY KEY,  
                          nombre VARCHAR(100));  
CREATE TABLE libro (id INT PRIMARY KEY,  
                     editorial_id INT REFERENCES editorial(id));
```

Tipos que no coinciden

List<Libro>, LocalDate, boolean frente a INT, VARCHAR, DATE. No hay correspondencia 1:1 automática.

Relaciones distintas

En Java, una referencia en memoria. En SQL, una clave foránea: hace falta un JOIN para reconstruirla.

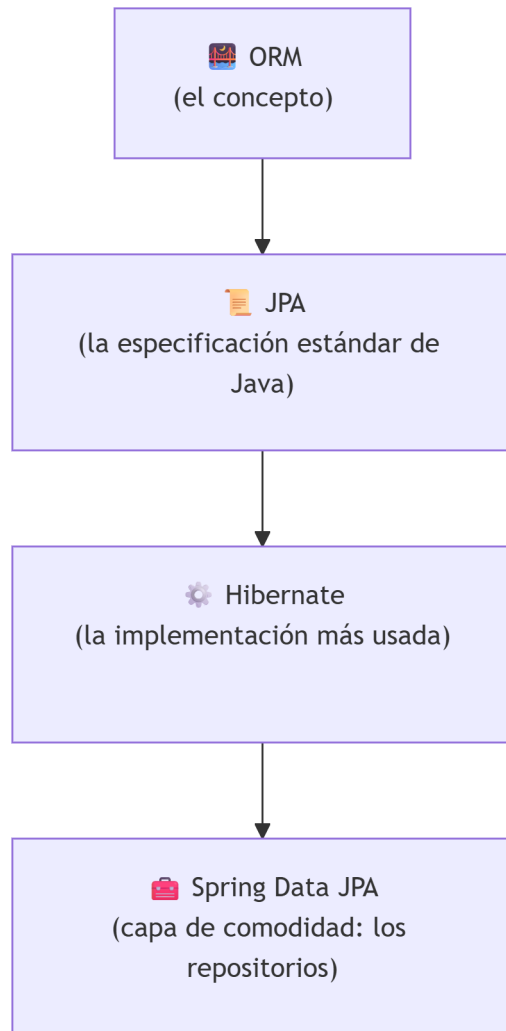
Identidad

En Java, == o equals(). En la base de datos, la clave primaria de la fila.

Desfase objeto-relacional (impedance mismatch)

No es un error de diseño de nadie: los dos mundos modelan la información de forma distinta por naturaleza.

La solución: una herramienta ORM



ORM

Declaras una sola vez la correspondencia entre una clase y una tabla; guardar, actualizar y consultar objetos se traduce solo al SQL.

JPA

La especificación estándar de Java: el contrato que dice qué anotaciones (@Entity, @Id, @OneToMany...) ofrece cualquier ORM. Como una interfaz.

Hibernate

Quien cumple ese contrato: la implementación de JPA más usada. Traduce cada anotación al SQL real que llega a PostgreSQL. Como la clase que implementa la interfaz.

Spring Data JPA

Una capa de comodidad sobre JPA/Hibernate: genera los repositorios (save, findById, findAll...). No es un ORM distinto.

Hibernate no es la única opción: EclipseLink es otra implementación completa de JPA, y fuera de Java cada lenguaje tiene la suya (SQLAlchemy en Python, Entity Framework en C#).

JDBC y los conectores de bases de datos

JDBC (Java Database Connectivity)

API estándar: una forma común de acceder a bases de datos relacionales desde Java, sin aprender una API distinta por cada gestor.

Cada gestor necesita un conector (driver) preparado para él:

PostgreSQL

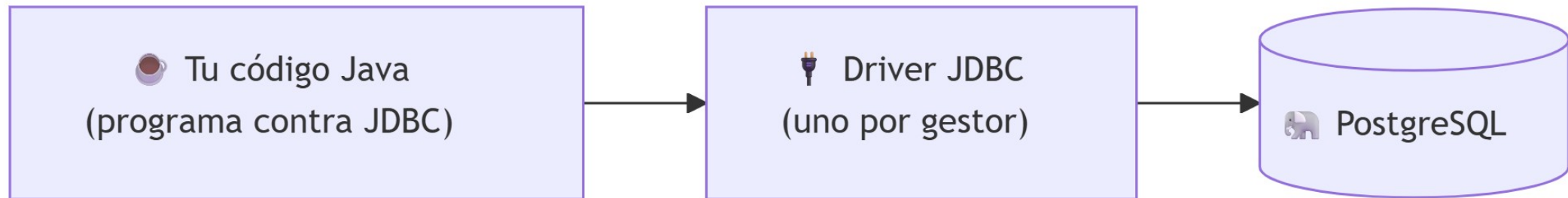
Su propio driver

MySQL

Otro driver

SQL Server

El suyo



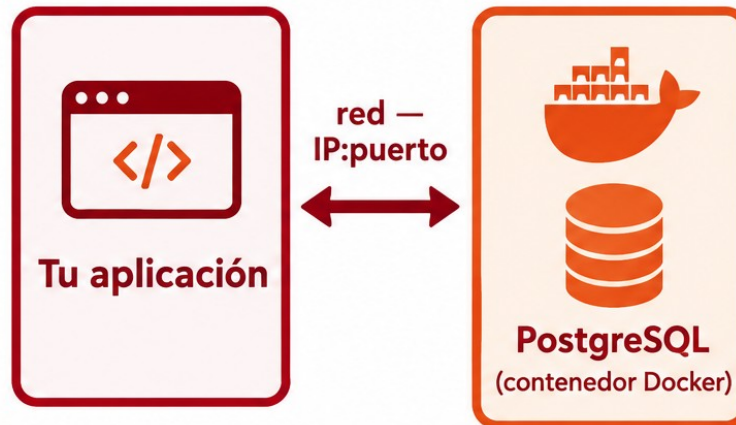
Si una aplicación pasara de PostgreSQL a MySQL, habría que cambiar el driver y la dirección de conexión, pero no aprender una API Java nueva.

Gestor embebido vs. independiente

Embebido



Independiente



Embebido

H2, SQLite. Dentro de tu propio proceso, sin nada que instalar. Tests rápidos, prototipos, apps de escritorio.

Independiente

PostgreSQL, MySQL, SQL Server. Servicio aparte, por IP y puerto, con muchos clientes a la vez. Aplicaciones reales.

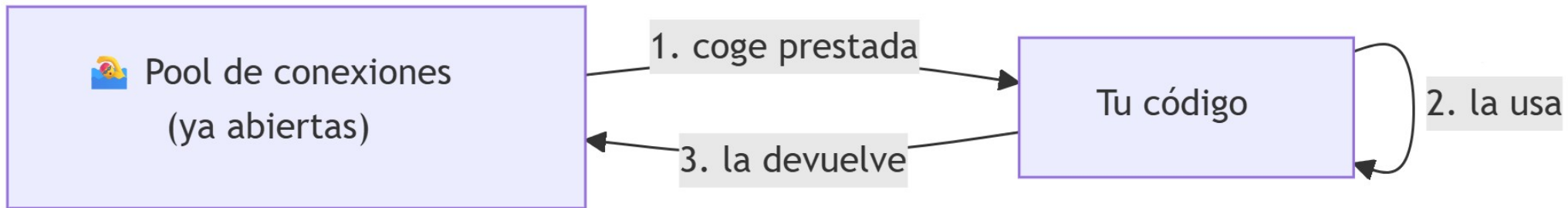
Pooling de conexiones

Abrir una conexión no es gratis

Negociación de red, autenticación, reserva de recursos: un coste real, medible en milisegundos.

Un pool las reutiliza

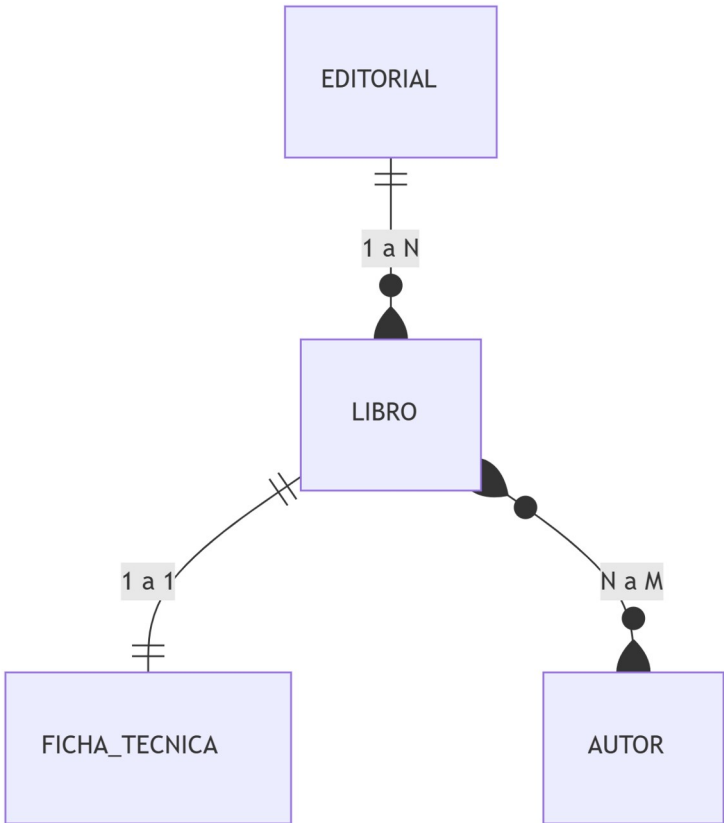
Mantiene un conjunto de conexiones ya abiertas y listas para usar, en vez de abrir una nueva cada vez.



Como taquillas ya abiertas: forzar una cerradura nueva cada vez que quieres guardar algo es lento. Coges una taquilla libre, la usas y la dejas lista para el siguiente.

Tipos de relación entre entidades

Multiplicidad (o cardinalidad): cuántas instancias de una clase se asocian con cuántas de la otra. Determina qué anotación JPA usas.



Relación	Ejemplo en la librería	Anotación
Uno a uno (1:1)	Un libro tiene una única ficha técnica	@OneToOne en los dos lados
Uno a muchos (1:N)	Una editorial publica muchos libros	@OneToMany + @ManyToOne
Muchos a muchos (N:M)	Un libro tiene varios autores y viceversa	@ManyToMany en los dos lados

Uno a uno: Libro y FichaTecnica

Libro: el lado inverso (mappedBy)

```
@Entity
public class Libro {
    @Id
    @GeneratedValue(strategy =
        GenerationType.IDENTITY)
    private Long id;

    @OneToOne(mappedBy = "libro",
        cascade = CascadeType.ALL)
    private FichaTecnica fichaTecnica;
}
```

mappedBy = "libro"

Señala el lado inverso: la clave foránea no vive aquí, sino en FichaTecnica.

Lo único que distingue un 1:1 de un 1:N en la base de datos es `unique = true`: sin él, varias fichas técnicas podrían apuntar al mismo libro.

FichaTecnica: la que tiene la clave foránea real

```
@Entity
public class FichaTecnica {
    @Id
    @GeneratedValue(strategy =
        GenerationType.IDENTITY)
    private Long id;
    private Integer numeroPaginas;
    private String idioma;

    @OneToOne
    @JoinColumn(name = "libro_id",
        unique = true)
    private Libro libro;
}
```

Muchos a muchos: Libro y Autor

Libro: con la tabla intermedia libro_autor

```
@Entity
public class Libro {
    @Id
    @GeneratedValue(strategy =
        GenerationType.IDENTITY)
    private Long id;

    @ManyToMany
    @JoinTable(
        name = "libro_autor",
        joinColumns = @JoinColumn(name = "libro_id"),
        inverseJoinColumns =
            @JoinColumn(name = "autor_id"))
    private List<Autor> autores;
}
```

Autor: el lado inverso

```
@Entity
public class Autor {
    @Id
    @GeneratedValue(strategy =
        GenerationType.IDENTITY)
    private Long id;
    private String nombre;

    @ManyToMany(mappedBy =
        "autores")
    private List<Libro> libros;
}
```

Ninguna de las dos tablas puede llevar la clave foránea: hace falta una tabla intermedia (libro_autor) con dos claves foráneas. @JoinTable le dice a Hibernate cómo se llama y cuáles son sus columnas.

Las piezas en el proyecto: gestor y conexión

docker-compose.yml: PostgreSQL independiente

```
services:
  postgres:
    image: postgres:18-alpine
    environment:
      POSTGRES_DB: libreria_db
      POSTGRES_USER: libreria_user
      POSTGRES_PASSWORD: password123
    ports:
      - "5432:5432"
```

application-dev.yml: la conexión

```
spring:
  datasource:
    url: jdbc:postgresql://
      localhost:5432/libreria_db
    username: libreria_user
    password: password123
```

spring.datasource.*

Todo lo que Spring Boot necesita:
URL JDBC (protocolo, host, puerto
y base de datos), usuario y
contraseña.

HikariCP

El pooling no lo configuras a
mano: Spring Boot lo trae por
defecto y se activa con el driver en
el pom.xml.

H2 en un perfil test

Mejora habitual: los tests simples
no dependen de tener Docker
levantado.

La estructura: la entidad Editorial

La estructura de la base de datos se declara sobre las propias clases Java, con anotaciones: el ORM en código real.

```
@Entity
@Table(name = "editorial")
public class Editorial {
    @Id
    @GeneratedValue(strategy =
        GenerationType.IDENTITY)
    private Long id;
    private String nombre;

    @OneToMany(mappedBy = "editorial",
        cascade = CascadeType.ALL,
        orphanRemoval = true)
    private List<Libro> libros;
}
```

1

@Entity + @Table

Esta clase se mapea contra la tabla editorial.

2

@Id + @GeneratedValue

El identificador y cómo se genera: autoincremental, delegado en la base de datos.

3

@OneToMany (mappedBy)

La clave foránea real vive en Libro (con su @ManyToOne de vuelta), no aquí.

cascade, orphanRemoval y ddl-auto

cascade = CascadeType.ALL

Propaga las operaciones de Editorial a sus Libro: si guardas o borras una editorial, Hibernate aplica lo mismo a todos sus libros.

orphanRemoval = true

Si quitas un libro de la lista sin borrar la editorial, queda «huérfano» y Hibernate lo elimina de la base de datos por su cuenta.

spring.jpa.hibernate.ddl-auto: update

Al arrancar, Hibernate crea o actualiza las tablas según esas anotaciones.

En producción casi nunca se usa update ni create-drop

Se prefieren migraciones controladas: Flyway o Liquibase.

Ideas clave

1

El desfase objeto-relacional es una diferencia de naturaleza entre objetos y referencias (Java) y tablas y claves foráneas (SQL).

2

Un ORM resuelve ese desfase: JPA es el estándar, Hibernate su implementación más usada y Spring Data JPA añade los repositorios.

3

JDBC es la API estándar de Java para bases de datos; un driver la implementa para cada gestor.

4

Embebido (H2, SQLite) corre dentro de tu proceso; independiente (PostgreSQL, MySQL), como servicio aparte por red.

5

El pooling reutiliza conexiones ya abiertas; en Spring Boot lo gestiona HikariCP por defecto.

6

Multiplicidad: 1:1 (`@OneToOne`), 1:N (`@OneToMany`/`@ManyToOne`) y N:M (`@ManyToMany` con `@JoinTable`).

7

En el curso: PostgreSQL independiente vía Docker; la conexión, en `application-dev.yml` (`spring.datasource.*`).

8

`mappedBy` señala el lado inverso, `cascade` propaga operaciones, `orphanRemoval` borra huérfanos y `ddl-auto` controla las tablas.